INFRAFLOW

PLATFORM ASSESSMENT REPORT

# Where Infraflow stands: architecture, incidents, and direction

A retrospective review of the visual AWS infrastructure builder — what the platform does, what broke during hands-on development and why, and whether its current trajectory is the right one.

PREPARED
**26 July 2026**

SCOPE
**Full-stack platform review**

BASIS
**Live codebase & production data**

SECTION 1

# 01 Executive summary

Infraflow is a visual, drag-and-drop builder for AWS infrastructure that generates real Terraform, validates it in three independent layers before anything touches AWS, and deploys it either directly or through a GitHub Actions pipeline. The core idea is sound and the safety engineering around it is genuinely more mature than most early-stage infrastructure tools. The rough edges found during this review are real, mostly operational rather than architectural, and each has a clear fix.

| 44 | 116 | 119 | 60 | 9 |
|---|---|---|---|---|
| AWS RESOURCE TYPES MODELED | DEPLOYMENTS RUN TO DATE | DIAGRAMS CREATED | SUCCESSFUL TEARDOWNS | REGISTERED USERS |

**Headline finding:** of 116 deployments, 44 ended in **failed** status. Investigation showed almost none of these were caused by the visual builder generating bad infrastructure — the validation pipeline catches that class of error reliably. Nearly every failure traced back to *operational* causes: exhausted local disk, a hard-delete that orphaned AWS account references, missing IAM permissions on the user's own connected role, or environment configuration drift between local and cloud. That distinction matters — it means the product's core logic is trustworthy, and the fixes needed are about hardening the edges around it, not rethinking the design.

SECTION 2

## 02 What the application actually is

Infraflow lets a user drag AWS services onto a canvas, wire them together, and get a deployable stack out the other end — without hand-writing Terraform. The stack is broader than a typical hackathon-scale IaC tool:

### Frontend

React + Vite, deployed as a static bundle to S3 behind CloudFront. A node-based visual canvas (React Flow) drives a Properties Panel that edits per-resource configuration, with live validation surfaced inline. Environment-aware build config (local dev talks to a local API via a dev-server proxy; the production build calls a Lambda-backed API Gateway directly).

### Backend

Node/Express, runnable two ways: as a conventional long-lived process (local development) or wrapped for AWS Lambda behind API Gateway (production), sharing the same route/controller code in both. MongoDB (Atlas in production, local in development) is the system of record for users, diagrams, deployments, AWS account connections, and application pipelines.

### Terraform generation & execution

Diagrams are compiled into real HCL by a generator that understands 44 AWS resource types, their required fields, and the connections between them (an API Gateway node wired to a Lambda node auto-generates the integration, route, and invoke permission; a Lambda wired to an IAM Role node references that role's ARN directly). Execution happens either by shelling out to the Terraform CLI locally, or by dispatching a GitHub Actions workflow — the same safety and status-tracking logic is shared between both executors rather than duplicated.

### Beyond infrastructure: application pipelines

A second system generates GitHub Actions CI/CD workflows for deploying the *application code* that runs on top of the infrastructure (React apps to S3/CloudFront, containers to ECS, Lambda functions), including automatic OIDC IAM role provisioning so no long-lived AWS keys sit in GitHub secrets.

### Administration

A Super Admin console covers user/role/credit management and, as of this review, a structured architecture changelog with impact ratings and a linked technical backlog — infrastructure for tracking the platform's own evolution.

SECTION 3

## 03 What's genuinely working well

These aren't table-stakes features — they're the kind of defensive engineering that usually only shows up after a platform has been burned by its absence at least once. Infraflow has them from early on.

### Three-layer validation before anything touches AWS

Design-time field checks, structural HCL checks, then a real `terraform validate` against provider schemas — proven during this review to catch errors (like an out-of-range Lambda memory value) that pure JavaScript validation structurally cannot.

### Safety-first failure handling

A failed first-time deploy auto-destroys anything it managed to create, so nothing orphans silently. A failed *update* deliberately does **not** auto-destroy — because that could tear down infrastructure that was working before the update started. That distinction is easy to get wrong and Infraflow gets it right.

### Storage abstracted from day one

A single storage-adapter interface (local disk vs. S3) means the same code path that runs Terraform locally today can run from Lambda tomorrow without a rewrite — which is exactly what happened during this review.

### Restart reconciliation

If the backend process dies mid-deployment, a startup routine finds anything left in an active state and resolves it conservatively — resuming a GitHub Actions run if possible, otherwise flagging for manual verification rather than guessing.

The Terraform-generation layer itself also deserves credit: connection-driven wiring (draw an edge, get a correct IAM trust relationship or API Gateway integration) is a genuinely differentiated feature relative to most visual IaC tools, which usually stop at generating isolated resource blocks.

SECTION 4

# 04 Challenges encountered

The following were found and fixed through direct, hands-on operation of the platform against real AWS infrastructure — not a code audit. Each is real, each recurred or nearly recurred, and each is now addressed.

| ISSUE | SEVERITY | ROOT CAUSE | RESOLUTION |
|---|---|---|---|
| **Disk exhaustion crashed the database** | HIGH | Every `terraform init` extracts the ~150MB AWS provider into a Windows temp folder that Terraform never cleans up on its own. Left unattended, this silently filled the system drive to 0 bytes free, which caused MongoDB's own diagnostics writer to fail — and MongoDB is designed to hard-abort rather than run in that state. | Reclaimed the accumulated space; the underlying accumulation pattern is now understood and documented for monitoring. |
| **Orphaned AWS account references** | HIGH | Disconnecting an AWS account permanently deleted its database record. Every deployment referencing that record — **all 116** in this workspace — became unable to resolve its AWS account on the next action, surfacing as an opaque "account not found" error. | Disconnect now soft-deletes (status flip, not row deletion); reconnecting the same account correctly revives the same record instead of creating an orphaning duplicate. |
| **Stuck IAM role blocked teardown** | HIGH | The connected AWS role was missing `iam:ListInstanceProfilesForRole`, a permission AWS requires before it will delete a role. Destroys partially completed — real resources removed from AWS — then got permanently stuck on the one resource the role couldn't delete. | Diagnosed and supplied the complete, correct IAM policy (this exact permission was already documented in the codebase but never surfaced in the product itself). |
| **Lambda deployment misconfigured for its own runtime** | MEDIUM | The platform's own backend was pointed at its conventional server entrypoint (`server.js`, built around `app.listen()`) instead of the Lambda-specific handler that already existed in the codebase. | Repointed to the correct handler; confirmed live end-to-end including database connection reuse across warm invocations. |
| **Cross-origin requests silently blocked** | MEDIUM | Once the frontend (CloudFront) and backend (API Gateway) became genuinely separate origins, the browser's own CORS policy blocked requests unless the backend explicitly allowlisted the frontend's domain. | Identified and resolved as a configuration step, not a code defect. |

| ISSUE | SEVERITY | ROOT CAUSE | RESOLUTION |
|---|---|---|---|
| **False-positive validation failure on Lambda code uploads** | MEDIUM | The validation layer that runs `terraform validate` did so in an empty scratch directory that never received the uploaded Lambda zip — so any diagram with an uploaded Lambda package failed validation, even when the code was completely correct. | The validator now stages the same artifact the real deploy would use before validating. |
| **Every deployment shares one generic name** | LOW | Every record in the list is labeled "Current visual infrastructure deployment," which made it easy to link a downstream system (an application pipeline) to the wrong backend entirely, since nothing in the UI distinguished a Lambda/API backend from a static-site frontend. | Flagged; not yet fixed. Root cause of at least one other incident in this table. |
| **Stale on-disk artifacts from earlier architecture** | LOW | An upload-storage refactor left duplicate files behind at the old, pre-refactor location — dead weight that nothing in the running application reads or writes anymore. | Removed; a reference-counted sweep now runs on a schedule for the artifact types this applies to going forward. |

SECTION 5

# 05 Risk areas and technical debt

## Configuration is scattered across three systems with no single source of truth

A working production deployment currently requires coordinated environment variables set correctly in three separate places — the Lambda console, the GitHub repository's secrets, and the GitHub repository's variables — with no in-product validation that they're mutually consistent. Nearly every "medium" severity issue in Section 4 traces back to this. There is no startup check that says *"your GitHub callback URL still points at localhost."*

## No evidence of automated testing

Every fix in this review was verified by hand — live HTTP calls, a real browser session, or direct database inspection — because there is no automated test suite to run instead. That's a reasonable trade-off at the current stage, but the bug classes found (a stale scratch directory, a validator missing a staging step, a cascading CSS layout constraint) are exactly the kind that a small integration-test suite catches automatically and cheaply going forward.

## Single AWS account for all testing

All 116 deployments in this workspace ran against one connected AWS account. That makes destructive operations (destroy, force-destroy) inherently higher-stakes to test, and likely explains why the IAM permission gap in Section 4 wasn't caught until a real teardown hit it.

## Frontend bundle size

The production build produces a single ~1.5MB JavaScript chunk, past the point where code-splitting would improve initial load time — a normal growing pain for a canvas-heavy application, not urgent, but worth planning for.

SECTION 6

## 06 Is this going in the right direction?

### Yes — with a caveat about pace, not direction.

The architectural decisions made early — separating validation into independent layers, treating "update" and "create" failures as different safety problems, abstracting storage so the same code can run locally or on Lambda, sharing safety logic between two structurally different execution backends instead of duplicating it — are the decisions that are hard to retrofit later and easy to get wrong. Infraflow got them right from early on. That's the strongest signal available that this is being built by people who understand the failure modes of infrastructure tooling, not just its happy path.

The caveat: most of the activity captured in this review was reactive — finding and fixing problems as they surfaced in live use, rather than problems caught ahead of time by tests or process. That's completely normal for a platform at this stage, and every fix made was a real, durable improvement, not a patch. But it's worth naming honestly: the ratio of "firefighting" to "planned building" in this review was high. Section 5's recommendations are specifically aimed at shifting that ratio.

The product thesis itself — a visual builder that won't let you generate infrastructure it hasn't verified, deployed through a pipeline that treats "something went wrong" as a first-class case instead of an edge case — is sound and, based on this review, differentiated from tools that generate Terraform and leave verification to the user. Nothing found during this review suggests the foundation needs to change. Everything found needs the edges tightened.

SECTION 7

# 07 Recommendations

1. **Centralize environment configuration validation**

   A single startup check (or an admin-console panel) that confirms the Lambda's env vars, the GitHub repo's secrets, and the GitHub repo's variables are mutually consistent — catching exactly the class of "works locally, silently broken in production" issue that recurred most often in this review.

2. **Surface the required IAM policy inside the product**

   The correct, complete policy already exists in the codebase but is never shown to the user connecting an AWS account. A "view/copy required policy" affordance on the Connect AWS page would have prevented the stuck-teardown incident entirely.

3. **Give deployments meaningful names**

   Auto-derive a name from what's actually on the canvas (service types present, or a required user-entered label) instead of a generic default — directly prevents the class of mistake where a downstream pipeline gets linked to the wrong backend.

4. **Add a small integration-test suite around the deployment lifecycle**

   Not full coverage — just the highest-value path (create → validate → deploy → update → destroy against a mocked or sandboxed AWS) to catch the class of regression that otherwise waits for a human to hit it in production.

5. **Monitor disk usage and orphan accumulation proactively**

   The provider-cache and upload-artifact cleanup built during this review are reactive fixes for problems that had already grown large. The same class of sweep, run on a schedule with alerting rather than discovered manually, closes the loop.

6. **Code-split the frontend bundle**

   Not urgent, but the canvas/builder, the admin console, and the application-pipeline UI are natural, low-risk split points before the single-chunk size becomes a real load-time problem.